

Mini-Projet 1 – PasswordCracker v1

Mise en œuvre du patron Simple Factory

1. Contexte

Dans le domaine de la cybersécurité, les mots de passe ne sont généralement pas stockés en clair dans les bases de données. Ils sont transformés à l'aide de fonctions de hachage cryptographiques.

Lors d'un audit de sécurité, il peut être nécessaire de vérifier la robustesse des mots de passe utilisés en tentant de retrouver un mot de passe à partir de son empreinte (hash).

L'objectif de ce mini-projet est de développer une première version d'un outil nommé **PasswordCracker** permettant de retrouver un mot de passe à partir de son hash MD5.

Cette première version devra être conçue en utilisant le patron de création **Simple Factory**.

2. Objectifs pédagogiques

À l'issue de ce mini-projet, vous devrez être capables de :

- Concevoir une architecture orientée objet modulaire ;
 - Utiliser le polymorphisme ;
 - Mettre en œuvre le patron Simple Factory ;
 - Développer une application en ligne de commande ;
 - Comprendre les avantages et les limites d'une fabrique simple.
-

3. Présentation du logiciel

Le programme développé s'appelle **passwordCracker**. Il reçoit :

- une méthode de cassage
 - BRUTE pour une attaque par brute force
 - DICO pour celle par dictionnaire
- un hash MD5

Exemples :

```
passwordCracker -m BRUTE -h e7247759c1633c0f9f1485f3690294a9
```

```
passwordCracker -m DICO -h e7247759c1633c0f9f1485f3690294a9
```

Résultat attendu :

```
Password found: test
```

ou

```
Password not found
```

Toute information pertinente pourra également être affichées (temps d'exécution, nombre de tentatives, etc.)

4. Fonctionnalités attendues

4.1 Cassage par dictionnaire

Le programme charge un dictionnaire contenant une liste de mots.

Exemple :

```
bonjour  
secret  
admin  
password  
azerty
```

Pour chaque mot :

1. Calculer son hash MD5 ;
2. Comparer au hash recherché ;
3. Retourner le mot correspondant.

4.2 Cassage par force brute

Le programme génère automatiquement toutes les combinaisons possibles.

Alphabet :

abcdefghijklmnopqrstuvwxyz

Longueur maximale :

4 caractères

Exemples :

a
b
...
aa
ab
...
test

Chaque combinaison doit être testée jusqu'à trouver une correspondance.

5. Architecture imposée

5.1 Interface commune

Toutes les stratégies doivent implémenter l'interface suivante :

```
public interface HashCracker {  
    String crack(String hash);  
}
```

La méthode retourne :

- le mot de passe trouvé ;
- null si aucun résultat n'est obtenu.

5.2 Stratégies concrètes

DictionaryHashCracker

Recherche dans un dictionnaire.

BruteForceHashCracker

Recherche exhaustive.

5.3 Fabrique simple

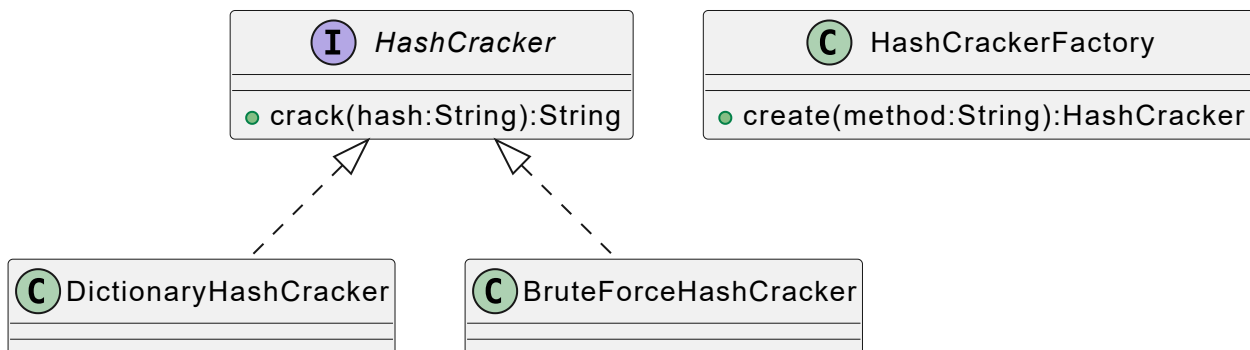
Les objets doivent être créés à travers une fabrique unique.

```
public class HashCrackerFactory {  
  
    public static HashCracker create(String method) {  
        ...  
    }  
  
}
```

Exemple :

```
HashCracker cracker = HashCrackerFactory.create("DICO");
```

6. Diagramme UML attendu



7. Contraintes

Les règles suivantes doivent être respectées :

- Le programme doit être écrit en Java.
- Les classes concrètes ne doivent pas être instanciées directement dans le programme principal.
- La création des objets doit être centralisée dans la fabrique.
- Les duplications de code doivent être évitées.

8. Travail demandé

Partie 1 : Analyse & Conception

Produire :

- un diagramme UML ;
- une description des responsabilités des classes.

Partie 2 : Développement

Implémenter :

- les stratégies ;
- la fabrique ;
- l'application console.

Partie 3 : Validation

Tester les différentes stratégies sur plusieurs exemples.

9. Livrables

Le code du projet devra être hébergé dans un dépôt public sur GitHub.

Le fichier README du dépôt devra contenir le rapport technique contenant les sections suivantes :

1. Introduction ;
2. Présentation du problème ;
3. Architecture ;
4. Diagramme UML ;
5. Usage du patron Simple Factory ;
6. Résultats obtenus ;
7. Difficultés rencontrées ;
8. Conclusion.

La section **Résultats obtenus** devra également inclure, directement ou sous forme de lien, la vidéo de présentation de l'exécution de l'outil. Cette dernière devra durer au maximum dix minutes.

10. Questions de réflexion

1. Quels avantages apporte la fabrique simple ?
 2. Quels sont ses inconvénients ?
 3. Que faut-il modifier lorsqu'une nouvelle stratégie est ajoutée ?
 4. La fabrique respecte-t-elle le principe Open/Closed ?
-

Préparation du mini-projet suivant

Dans ce mini-projet, l'ajout d'une nouvelle stratégie nécessite la modification de la classe HashCrackerFactory.

Cette limitation sera étudiée et corrigée dans le mini-projet suivant.